

Amortized eFPGA Layout Search via Reinforcement Learning: Zero-Shot Generalization Across Circuit Benchmarks

Anonymous Author(s)

Abstract

eFPGA fabrics embed reconfigurability in fixed silicon for acceleration, post-fabrication updates, and IP security. We train a single graph-and-grid-conditioned RL policy that places H-blocks (DSP/BRAM) and selects the fabric aspect ratio across 11 circuits, then apply the same checkpoint zero-shot to circuits never seen in training. Jointly trained, the policy reaches 27.60% aggregate in-pool ADP reduction (three seeds); applied zero-shot to six held-out circuits (including fabrics up to $3.4\times$ the training area), every result is positive, averaging 31.23%, with markedly lower seed variance than in-pool. Against a GA sharing the same VTR oracle, metric, and baseline, the fine-tuned policy reaches a comparable best ADP reduction, while zero-shot already lands a positive result with no per-design search at all. Unlike per-instance search, the trained policy amortizes: each new netlist costs one rollout, not a restarted search.

1 Introduction

Embedding a small reconfigurable fabric into otherwise fixed silicon, an eFPGA, lets a chip adapt after fabrication, whether as a standalone ASIC or as an accelerator block inside a larger SoC: acceleration, protocol updates, IP security, and extended product life managed through the bitstream rather than a chip respin [20, 21]. A generic FPGA is over-provisioned for any single application: it contains extra routing and heterogeneous blocks (*H-blocks*: DSP slices, BRAMs) the deployed netlist never fully uses. An eFPGA fabric sized for one specific netlist can recover much of that overhead. The product of routing area, critical-path delay, and dynamic power (ADP throughout) quantifies this overhead directly, and minimizing it benefits every eFPGA application [3, 20].¹

Two prior papers show per-instance search substantially reduces eFPGA overhead: GOLDS [20] uses a GA with Area $\times$ Delay fitness; a follow-up applies NSGA-II and also co-evolves CLB LUT size and arithmetic-block sizing [3]. Both report gains on their own Area $\times$ Delay metric (Section 2), but both are *testbench-specific*: each run re-initializes from scratch, and nothing learned on one netlist transfers to the next: every new target pays the full search cost again.

We train a single graph-and-grid-conditioned RL policy to choose H-block placement and fabric aspect ratio, the problem where per-instance GA search has been the only option. Unlike RL methods that assign blocks to an already-fabricated grid, our policy *constructs* the fabric layout itself. Trained jointly on 11 circuits and scored by the real VTR flow, the same checkpoint applies zero-shot to unseen circuits, including fabrics up to $3.4\times$ the training pool area. Training runs once; each new netlist costs one rollout. We compare against a from-scratch GA under identical oracle, metric, and baseline: the fine-tuned policy reaches a comparable best ADP reduction, while

zero-shot already gives a positive result with no per-design search at all (Section 5.2).

This paper contributes:

- A graph-and-grid-conditioned RL policy for H-block placement and aspect-ratio selection, with one fixed-size checkpoint applied zero-shot to unseen circuits and evaluated against the true VTR flow rather than a proxy (Sections 3.3, 3.4).
- A comparison against a GA on held-out softmax under the same oracle, metric, and baseline (Section 5.2).
- An aggregate in-pool ADP reduction of 27.60% across three seeds (Section 5.1).
- A three-seed reliability analysis comparing in-pool and zero-shot seed sensitivity, reported as raw per-seed results (Sections 5.1, 5.3).

2 Background and Related Work

2.1 Per-Design eFPGA Layout

A generic island-style FPGA provisions H-blocks (DSP, BRAM) and routing for arbitrary workloads; any single application leaves much of this unused, paying area and delay for resources it never exercises. This motivates fitting tile placement and fabric dimensions to one specific netlist for a compact, efficient fabric. Applications include IP security (function absent from fixed silicon until configured [6, 21]), domain-specific acceleration, and post-fabrication reconfigurability [20]. Subsequent work reduces overhead by shrinking underutilized routing [15] or de-regularizing the fabric [8], both motivating the non-island-style layouts this paper synthesizes. All rely on per-design search; none amortizes layout effort across netlists. GOLDS [20] introduced layout search built for a single design: a GA over H-block placement with Area $\times$ Delay fitness through the real VTR flow. A follow-up [3] applies NSGA-II, also co-evolving CLB LUT size and H-block sizing. Both report per-design gains on their own two-term Area $\times$ Delay metric, a different quantity from the three-term ADP this paper reports. Both start from a random population for every new design, transferring nothing from prior runs. We target the same problem GOLDS defines with the same VTR oracle, enabling a clean controlled comparison (Section 5.2); block-sizing co-evolution is a natural extension (Section 6).

2.2 Learned Policies for Physical Design

AlphaChip [11, 18] trains an RL agent to place ASIC macros using a learned proxy and reports cross-design transfer: the amortization argument we make here, at a much larger scale. Subsequent work explores visual representations [17], joint placement-and-routing [5], and transformer policies [16], all supporting the claim that a trained policy amortizes layout cost across designs as per-instance search cannot. Unlike AlphaChip, we evaluate every candidate against the true VTR flow, not a proxy: affordable at our scale, where a VTR run takes seconds to low minutes (Section 4.1).

¹GOLDS [20] uses the same abbreviation, ADP, for a different, two-term Area $\times$ Delay product; ours is three-term (Area $\times$ Delay $\times$ Power) throughout this paper.

Within the FPGA setting, RL has guided placement onto an *already-fabricated* grid (annealing perturbations in VPR [10] or a Gym-style VPR environment [4]), but always assigns blocks to a fixed grid, never shapes it. Transferable EDA policies exist for tool tuning [13] and global placement [12]; our policy instead synthesizes the fabric geometry.

2.3 VTR/VPR Background

We build on the open-source Verilog-to-Routing (VTR) [9]: Yosys synthesizes the design [7] and VPR [2] packs, places, and routes against an architecture XML of tile types, positions, and connectivity. This XML (which GOLDS and NSGA-II generate by search and we generate by policy rollout) must represent arbitrary heterogeneous layouts, not just island-style grids, which is what makes this entire line of work possible.

3 Methodology

3.1 Problem Formulation

A fabric optimized for one netlist trades general-purpose flexibility for a significantly smaller, more efficient implementation: it accommodates parameter changes, post-fabrication fixes, and a known set of operational modes on that one primary function, not arbitrary workloads with substantially different H-block requirements.

Two roles are easily conflated: the *policy* designs the fabric; the *VTR toolchain* maps the circuit onto it. Given a netlist requiring n_{dsp} DSP and n_{bram} BRAM blocks, the policy emits

$$\pi = \left(\text{ar}, \{(x_i, y_i)\}_{i=1}^{n_{\text{dsp}}+n_{\text{bram}}} \right) : \quad (1)$$

a fabric aspect ratio $\text{ar} \in \mathcal{R}$ and a tile coordinate for every H-block. VTR then synthesizes, packs, places, routes, and auto-sizes the array to the smallest grid of the chosen aspect ratio that holds the design. The policy never places a CLB or fixes grid dimensions; it shapes the fabric and positions H-blocks; VTR reports back the realized cost. We take that cost to be the three-term ADP (Section 1):

$$\text{ADP}(\pi) = \text{Area}(\pi) \times \text{Delay}(\pi) \times \text{Power}(\pi), \quad (2)$$

where each term is from the VTR flow (Section 4). Area is VPR’s *routing area*: the physical tile-grid footprint including routing channels, which tracks the fabric geometry the policy controls and reflects actual on-chip die area; logical CLB area stays fixed regardless of layout. We report results as percentage ADP reduction relative to a traditional island-style baseline at matched logic capacity.

3.2 RL Formulation

We pose placement as a finite-horizon sequential decision process and train with Maskable PPO [14] (MaskablePPO), which restricts the action space to legal tile coordinates at each step.

Episode structure. Step 0 selects the fabric aspect ratio from $\mathcal{R}$; each subsequent step positions one H-block (BRAMs then DSPs, both ordered by descending shared-net connectivity). Once all blocks are placed, the environment renders the layout into an architecture XML via Jinja2 and invokes VTR to obtain $\text{Area}(\pi)$, $\text{Delay}(\pi)$, $\text{Power}(\pi)$; intermediate steps receive zero reward.

Action space. The action space is Discrete($W_{\text{max}} \times H_{\text{max}}$), fixed across all benchmarks (Section 3.4). Step-0 actions select from $\mathcal{R}$; later steps decode an index to (x, y) via a fixed stride on H_{max} .

Action masking restricts the policy to legal, non-overlapping, in-bounds coordinates for the active benchmark.

Reward. The terminal reward is a weighted sum of negative log-ratios against the traditional baseline for the active benchmark:

$$r = -\left(w_a \log \frac{\text{Area}}{\text{Area}_0} + w_d \log \frac{\text{Delay}}{\text{Delay}_0} + w_p \log \frac{\text{Power}}{\text{Power}_0} \right), \quad (3)$$

where $\text{Area}_0, \text{Delay}_0, \text{Power}_0$ are the traditional-baseline metrics and w_a, w_d, w_p are configurable (Section 3.5). With $w_a = w_d = w_p = 1$, $r = \log(\text{ADP}_0/\text{ADP})$, making reward and headline metric directly comparable. An invalid choice (degenerate aspect ratio, out-of-bounds or overlapping placement, or a VTR failure) terminates early with penalty -10 .

3.3 GCN Feature Extractor

To generalize across benchmarks with varying netlist sizes, H-block counts, and grid dimensions, we encode each netlist as a graph, giving the policy the inductive bias for cross-topology transfer that graph learning provides elsewhere in EDA [22]. We reduce each netlist to a compact fixed-schema graph: one node per H-block instance plus one aggregate “fabric” node for the remaining standard-cell logic, with node features [is_dsp, is_bram, is_fabric, net_count_normalized]. Edges are weighted by shared-net count, log-normalized to $[0, 1]$, both between H-blocks and between each H-block and the fabric node.

The graph is padded to a fixed maximum node and edge count (Section 3.4) and encoded by two width-64 graph convolution layers, giving every H-block a learned embedding. Global mean-pooling yields a whole-netlist embedding, and the embedding of the node under consideration at the current step yields a per-step embedding; both are concatenated with a CNN encoding of the 4-channel occupancy grid and the benchmark’s normalized fabric dimensions, then projected to a single 128-dimensional vector consumed by the PPO actor and critic heads. The encoder trains end-to-end under the PPO objective, with no separate pretraining stage.

3.4 Fixed-Capacity Sizing and Action Masking

Policy dimensions are fixed at construction, yet we want one checkpoint loadable against benchmarks never trained on. We therefore size every dimension-dependent quantity, the maximum fabric width and height and the maximum node and edge count, from a deliberate superset of benchmark sizes that includes ones reserved for zero-shot evaluation. Per-episode action masking restricts the fixed-size network to each benchmark’s smaller real footprint.

3.5 Training Setup

Training samples uniformly from the benchmark pool, episode-by-episode. We use MaskablePPO with 24 parallel environments, 48 rollout steps per update, batch size 144, 15 epochs per update, entropy coefficient 0.01, and reward weights $w_a = w_d = w_p = 1$ (Eq. 3), giving reward $= \log(\text{ADP}_0/\text{ADP})$. Each run trains for 13,000 episodes with a 300s VTR timeout per episode. We report three independent seeds (7, 42, 123) throughout Section 5 to characterize reliability rather than present a single run as ground truth.

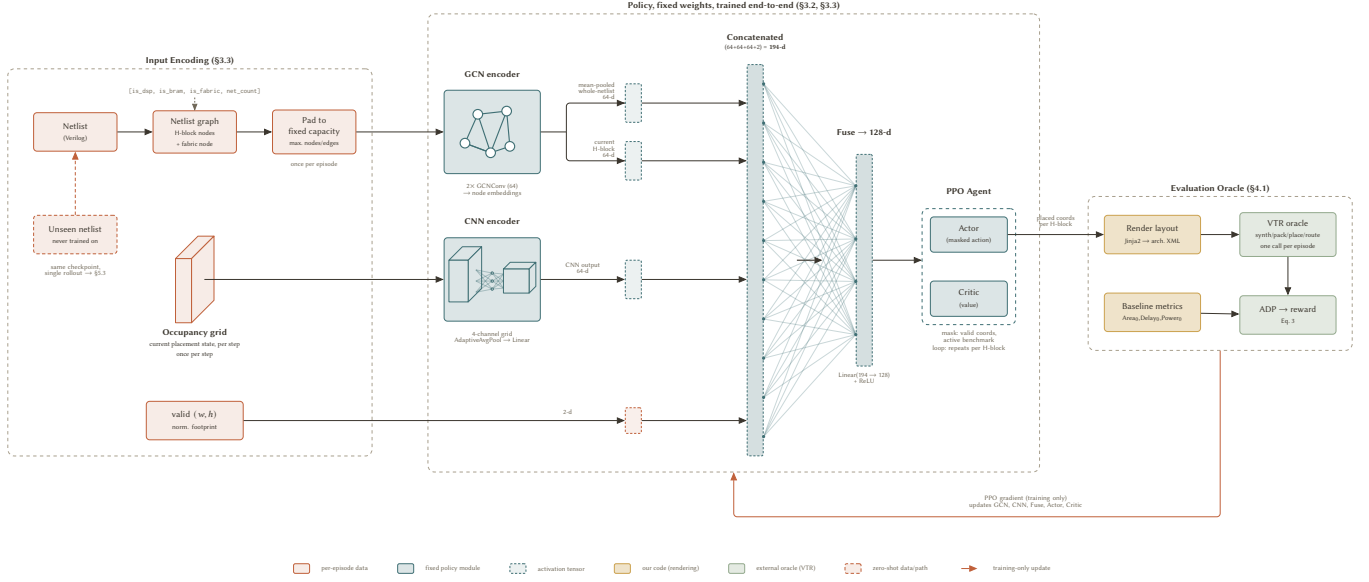


Figure 1: System overview: a netlist becomes a fixed-schema graph (Panel A), encoded jointly with the occupancy grid by a fixed-weight GCN+CNN policy that emits aspect ratio and block coordinates under action masking (Panel B); the policy decodes one H-block per step, updating the occupancy grid, until the full layout is placed. Our own code then renders that layout into an architecture XML, which the real, external VTR flow scores once per episode (Panel C), producing the reward (computed against the traditional baseline) that updates the policy (training only). The same trained weights accept any netlist that fits within the fixed-capacity superset, including ones never seen during training (dashed path, bottom-left): this is the zero-shot path evaluated in Section 5.3.

4 Experimental Setup

4.1 Evaluation Oracle

Every candidate placement is scored by the true toolchain, not a proxy. A placement is rendered into a VTR architecture XML and VPR constraints file via Jinja2 templates, then run through the full VTR flow (synthesis, packing, placement, routing, and power estimation against a 45nm PTM file), and routing area, critical-path delay, and dynamic power are extracted from VPR reports. All evaluations use VTR 9.0.0 (git 22a09d39, GCC 13.3.0). Evaluated pairs are cached; the synthesis-to-routing flow dominates cost (seconds to low minutes per call).

4.2 Benchmarks

Figure 2 lists all 17 benchmarks with grid size and H-block counts: 11 for training and 6 held-out for zero-shot evaluation only (Section 5.3), drawn from the VTR suite [19], Koios [1], and two small probes authored for this work, covering the small-fabric end the standard suites underrepresent. Four held-out benchmarks (softmax, reduction_layer, arm_core, mkDelayWorker32B) are larger circuits, probing fabric sizes well beyond the training pool. The GA comparison (Section 5.2) uses softmax, held out of RL training.

4.3 Baselines Compared

We compare against: (1) the traditional island-style eFPGA baseline²; and (2) a genetic algorithm implemented under the same evaluation oracle and ADP metric (Section 5.2).

5 Results

5.1 In-Pool Performance

Figure 2 (upper group) shows per-benchmark in-pool ADP reduction across three seeds. The pooled aggregate (summed ADP over all 11 benchmarks versus summed baseline) is 23.11%, 35.73%, and 23.97% for seeds 7, 42, and 123 respectively, averaging **27.60%**. Nine of eleven benchmarks are consistently positive and stable; raygentop is the only persistent failure ($[-2.2, -1.7]\%$ across all seeds). or1200 and boundtop are positive but highly seed-sensitive, spanning 28–29 points each and driving most of the aggregate variance. In-pool performance is considerably more seed-sensitive than zero-shot, which stays within ~ 12 points across all six held-out benchmarks (Section 5.3): early convergence on most training benchmarks leaves fewer episodes for slow-to-converge ones like or1200, so which seed handles them well varies run to run.

²VTR’s default island-style template (k6_frac_N10_mem32K_40nm: 6-input LUTs, cluster size 10, 32K-bit BRAMs, 40nm) at matched logic capacity, the same template GOLDS [20] uses.

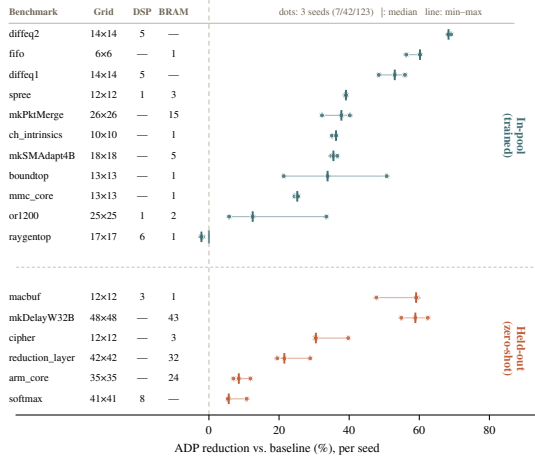


Figure 2: Per-seed ADP reduction for all 17 benchmarks (11 in-pool, 6 held-out zero-shot).

5.2 GA Comparison

For a fair point of comparison, we implement a GA under the same oracle, ADP metric, and baseline on softmax: roulette-wheel selection, single-point crossover, per-gene mutation over H-block coordinates and aspect ratio, population 8, capped at 500 generations with patience 50. The GA and the policy search in fundamentally different ways, a GA generation is not an RL episode, so we compare the one quantity that is directly comparable between them: the best layout each one finds. The GA’s best reaches **14.62%** ADP reduction. Zero-shot, the policy already achieves **5.39–10.77%** across seeds (7, 42, 123) with no search at all; fine-tuned on softmax from seed 42, it reaches **14.82%**, matching the GA’s best.

The GA pays its search cost again from scratch for every new design. The trained policy pays its cost once and reuses it: zero-shot lands a positive result before any softmax-specific search begins, and the advantage compounds with the number of designs the same checkpoint is later applied to.

Figure 3 shows the physical saving on softmax. The traditional fabric provisions BRAM columns the netlist never uses; the policy omits them and selects an aspect ratio under which VTR auto-sizes the logic to a 22% smaller array, the dominant ADP term. The policy never moves a CLB; it removes idle columns and reshapes the fabric, and the tighter routing is VTR’s.

5.3 Zero-Shot Generalization

Figure 2 (lower group) reports ADP reduction on six held-out benchmarks across the same three seeds, with zero benchmark-specific training. All six fit within the checkpoint’s fixed dimensions. Only two (softmax, reduction_layer) were used to choose those dimensions; the other four were sized in by benchmarks elsewhere in the sizing superset, a harder test of generalization. Four of the six are larger than anything in the training pool (up to 2,304 tiles vs. 676), making this size extrapolation, not interpolation. Every benchmark is positive under every seed (no sign flips), and every per-seed range

is narrow (≤ 12 points), far from the or1200/boundtop spread. The overall average is **31.23%**.

6 Conclusion

This paper trains one policy across 11 circuits and compares it to a GA under the same oracle, metric, and baseline (Section 5.2): the fine-tuned policy reaches a similar best ADP reduction, and zero-shot gives a positive result with no per-design search. The same policy applies zero-shot to six held-out benchmarks, four larger than anything in training (Section 5.3).

These results rest on three seeds and one deterministic rollout per zero-shot benchmark, with in-pool fitting showing more seed sensitivity than zero-shot transfer within that sample (Section 5.1). A few directions follow naturally: a systematic stochastic-rollout evaluation; extending the action space, currently placement and aspect ratio only, to co-evolve CLB LUT size and block sizing [3]; and running on the full GOLDS/NSGA-II benchmark set for direct comparison.

References

- [1] Aman Arora, Andrew Boutros, Daniel Rauch, Aishwarya Rajen, Aatman Borda, Seyed A. Damghani, Samidh Mehta, Sangram Kate, Pragnesh Patel, Kenneth B. Kent, Vaughn Betz, and Lizy K. John. 2021. Koios: A Deep Learning Benchmark Suite for FPGA Architecture and CAD Research. In *31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE. doi:10.1109/FPL53798.2021.00010
- [2] Vaughn Betz and Jonathan Rose. 1997. VPR: A New Packing, Placement and Routing Tool for FPGA Research. In *7th International Workshop on Field-Programmable Logic and Applications (FPL)*. Springer-Verlag, 213–222. doi:10.1007/3-540-63465-7_226
- [3] D. V. Bhargava, G. Pradyumna, and Madhav Rao. 2025. Meta-Heuristic Optimization of Custom Heterogeneous Blocks Defined eFPGA Design. In *38th International Conference on VLSI Design (VLSID)*. IEEE, 326–331. doi:10.1109/VLSID64188.2025.00069
- [4] Ruichen Chen, Shengyao Lu, Mohamed A. Elgammal, Peter Chun, Vaughn Betz, and Di Niu. 2023. VPR-Gym: A Platform for Exploring AI Techniques in FPGA Placement Optimization. In *33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 72–78. doi:10.1109/FPL60245.2023.00018
- [5] Ruoyu Cheng and Junchi Yan. 2021. On Joint Learning for Solving Placement and Routing in Chip Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 16508–16519.
- [6] Luca Collini, Jitendra Bhandari, Chiara Muscarello, Abdoul Khader Thakattou Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2025. ARIANNA: An Automatic Design Flow for Fabric Customization and eFPGA Redaction. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* (2025). volume/issue/pages unverified at press time. doi:10.1145/3737287
- [7] Seyed Alireza Damghani and Kenneth B. Kent. 2022. Yosys+Odin-II: The Odin-II Partial Mapper with Yosys Coarse-grained Netlists in VTR. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 157. doi:10.1145/3490422.3502344
- [8] Voktho Das, Kimia Zamiri Azar, and Hadi M. Kamali. 2026. NuRedact: Non-Uniform eFPGA Architecture for Low-Overhead and Secure IP Redaction. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE.
- [9] Mohamed A. Elgammal, Amin Mohaghegh, Soheil G. Shahrour, Fatemehsadat Mahmoudi, Fahrican Kosar, Kimia Taleai, Joshua Fife, Daniel Khadivi, Kevin E. Murray, Andrew Boutros, Kenneth B. Kent, Jeffrey B. Goeders, and Vaughn Betz. 2025. VTR 9: Open-Source CAD for Fabric and Beyond FPGA Architecture Exploration. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 18, 3 (2025), 39:1–39:35. doi:10.1145/3734798
- [10] Mohamed A. Elgammal, Kevin E. Murray, and Vaughn Betz. 2022. RLPlace: Using Reinforcement Learning and Smart Perturbations to Optimize FPGA Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 41, 8 (2022), 2532–2545. doi:10.1109/TCAD.2021.3109863
- [11] Anna Goldie, Azalia Mirhoseini, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2024. Addendum: A graph placement methodology for fast chip design. *Nature* 634, 8034 (2024), E10–E11. doi:10.1038/s41586-024-08032-5

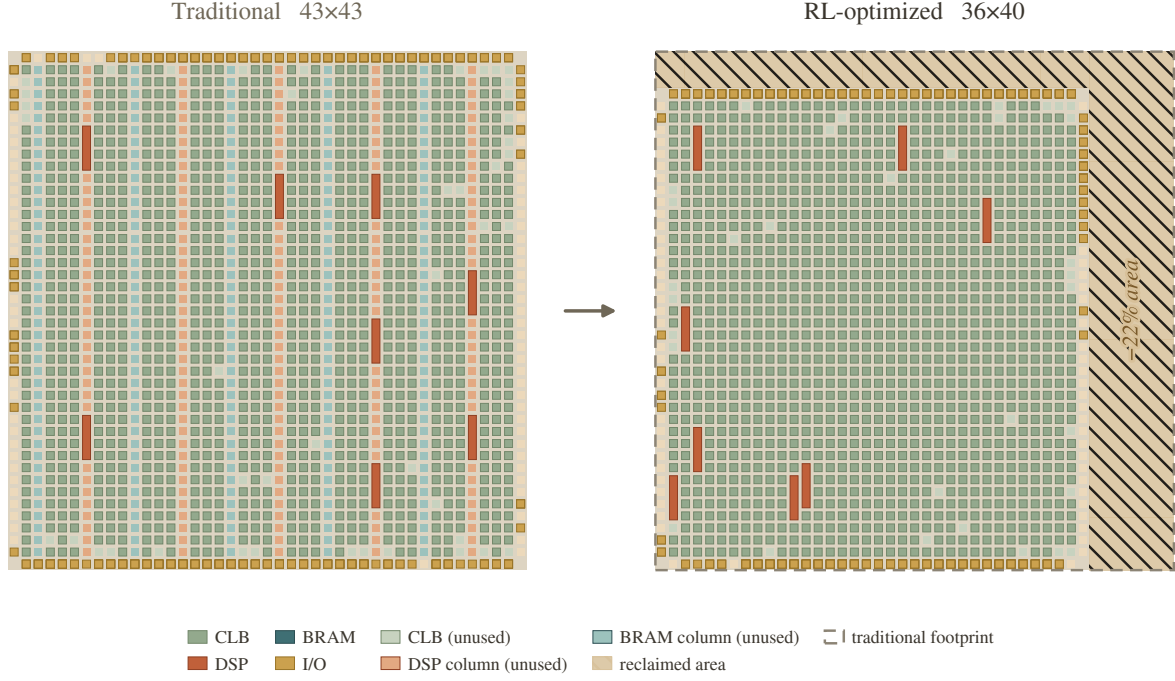


Figure 3: What the policy recovers, physically (softmax, the GA comparison benchmark). Both fabrics are drawn at the same tile scale. The traditional island-style baseline (left) is a 43×43 heterogeneous fabric that provisions BRAM columns (plum) the netlist never uses, with DSP blocks (terracotta) spread across regular DSP columns. The RL-optimized fabric (right) tailors the architecture to the netlist: it drops the unused BRAM columns entirely and repositions the DSPs, under an aspect ratio that lets VTR auto-size the same logic down to 36×40 . The dashed outline is the traditional footprint and the hatched strip is the reclaimed area: a 22% smaller fabric, the dominant term in the 15% ADP reduction. Gaps between tiles are routing channels.

- [12] Yunbo Hou, Haoran Ye, Shuwen Yang, Yingxue Zhang, Siyuan Xu, and Guojie Song. 2025. TransPlace: Transferable Circuit Global Placement via Graph Neural Network. *arXiv preprint arXiv:2501.05667* (2025).
- [13] Hao-Hsiang Hsiao, Yi-Chen Lu, Pruek Vanna-Iampikul, and Sung Kyu Lim. 2024. FastTuner: Transferable Physical Design Parameter Optimization using Fast Reinforcement Learning. In *International Symposium on Physical Design (ISPD)*. ACM.
- [14] Shengyi Huang and Santiago Ontañón. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the 35th International FLAIRS Conference*, Vol. 35. doi:10.32473/flairs.v35i.130584
- [15] Hadi M. Kamali, Kimia Z. Azar, Farimah Farahmandi, and Mark Tehranipoor. 2023. SheLL: Shrinking eFPGA Fabrics for Logic Locking. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–6.
- [16] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. 2023. ChiPFormer: Transferable Chip Placement via Offline Decision Transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML)* (PMLR, Vol. 202). 18346–18364.
- [17] Yao Lai, Yao Mu, and Ping Luo. 2022. MaskPlace: Fast Chip Placement via Reinforced Visual Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 24019–24030.
- [18] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212. doi:10.1038/s41586-021-03544-w
- [19] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, Mohamed Eldafrawy, Jean-Philippe Legault, Eugene Sha, Aaron G. Graham, Jean Wu, Matthew J. P. Walker, Hanqing Zeng, Panagiotis Patros, Jason Luu, Kenneth B. Kent, and Vaughn Betz. 2020. VTR 8: High-performance CAD and Customizable FPGA Architecture Modelling. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 13, 2 (2020), 9:1–9:55. doi:10.1145/3388617
- [20] Pratyush Nandi, Anubhav Mishra, and Madhav Rao. 2024. GOLDS: Genetic Algorithm-based Optimization of Custom FPGA Architecture Layout Design for Secure Silicon. In *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*. ACM, 92–97. doi:10.1145/3649476.3658743
- [21] Chiara Muscari Tomajoli, Luca Collini, Jitendra Bhandari, Abdul Khader Thakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2022. ALICE: An Automatic Design Flow for eFPGA Redaction. In *59th ACM/IEEE Design Automation Conference (DAC)*. 781–786. doi:10.1145/3489517.3530543
- [22] Nan Wu et al. 2023. GAMORA: Graph Learning based Symbolic Reasoning for Large-Scale Boolean Networks. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.